Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes

TD : Ingénierie Dirigée par les Modèles

Transformation (génération de code) par template avec Acceleo

Du modèle `Library.ecore` au code Python, SQL, Java et Markdown

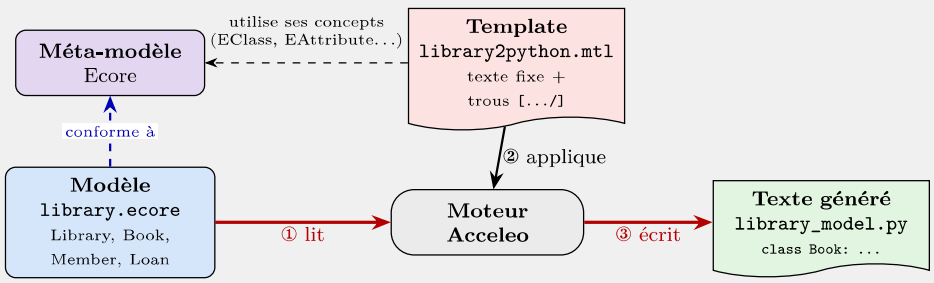
(Énoncé)

Pr.Aissam BERRAHOU
Année universitaire 2026–2027

Objectifs et consignes

1. Expliquer le principe d'une transformation **modèle → texte** (M2T) **par template**.
2. Distinguer le **méta-modèle** (Ecore) du **modèle** d'entrée (`library.ecore`).
3. **Naviguer** dans un modèle Ecore avec des expressions OCL (`.`, `->select`, `->filter`, `->size`...).
4. **Lire** un template Acceleo et **prédire** exactement le texte produit.
5. **Écrire** des templates et des queries (Python, SQL, Java, Markdown) et **corriger** un template erroné.

Le TD suit le **fil rouge du cours** (Lecture 3) et du **tutoriel complet Acceleo** : le modèle `library.ecore`. Les exercices se font **sur papier** ; le TP associé permet d'exécuter les mêmes templates dans Eclipse. On utilise la syntaxe **Acceleo 3** (expressions OCL). L'aide-mémoire du cours est autorisé.

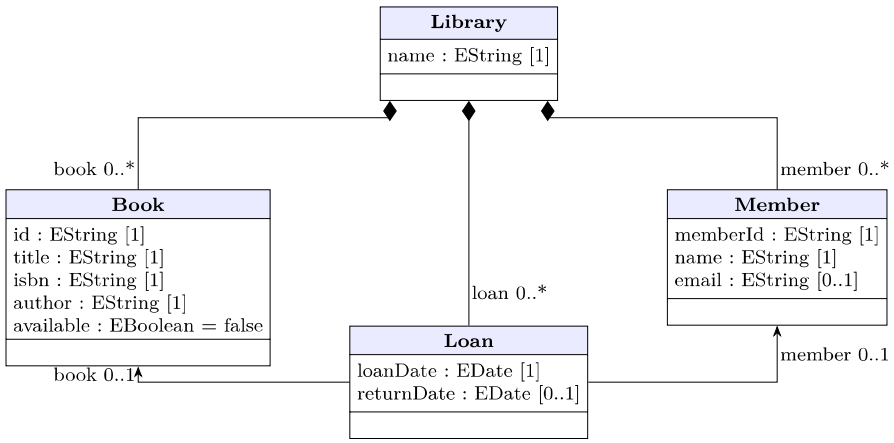


Plan du TD :

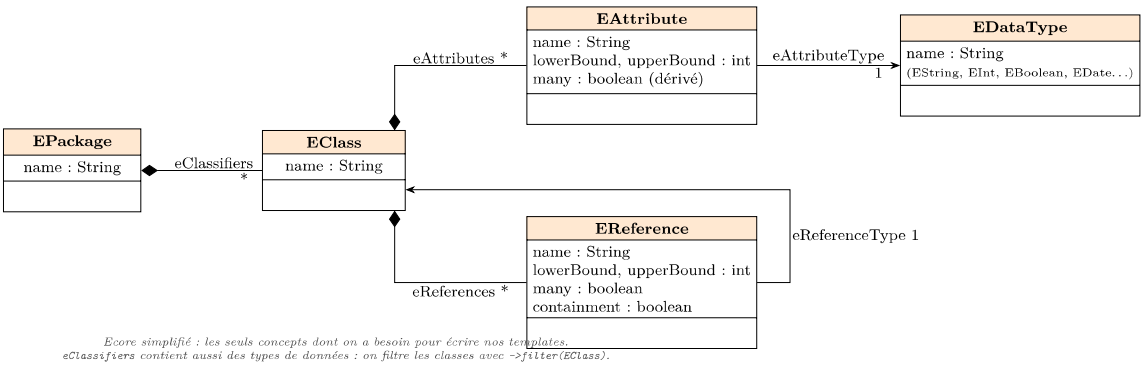
Exercice	Thème	Notion principale
1	Vrai ou faux ?	Principe du template [Simple]
2	Modèle ou méta-modèle ?	Ecore, URI, `EPackage` [Simple]
3	Naviguer dans `library.ecore`	Expressions OCL [Simple]
4	Prédire le résultat d'un template	`[for]`, `separator`, `[if]` [Simple]
5	Compléter le générateur Python du tutoriel	Boucles imbriquées, query [Moyen]
6	Queries : la connaissance de la plateforme	`[query]`, `if then else` [Moyen]
7	Trouver et corriger les erreurs	Syntaxe Acceleo [Moyen]
8	Générer le script SQL	Clés primaires et étrangères [Moyen]
9	Générer les classes Java	`[file]` par classe, sous-templates [Avancé]
10	Documentation Markdown et zones protégées	Autre cible, régénération [Avancé]

Rappel : le modèle utilisé

Le modèle `library.ecore` (paquetage `library`) a été dessiné avec Ecore Tools. Les attributs notés `[1]` sont obligatoires (`lowerBound = 1`) ; `[0..1]` sont facultatifs (`lowerBound = 0`). Les losanges noirs sont des **compositions** (`containment = true`).



Rappel du méta-modèle **Ecore** simplifié (les seuls concepts utiles ici) :



Exercice 1 : Vrai ou faux ? [Simple]

Répondre par **vrai** ou **faux** et **corriger** les affirmations fausses.

A1. Acceleo transforme un modèle en un autre modèle conforme à un méta-modèle cible.

A2. Un template est un texte dans lequel les parties variables sont écrites entre crochets.

A3. L'expression `[cls.name]` écrit le nom de la classe `cls`.

A4. Le template `library2python.mtl` ne fonctionne que pour le modèle `library.ecore`.

A5. Avec `separator(', ')`, une virgule est écrite après chaque élément, y compris le dernier.

A6. Un bloc `[file]` placé à l'intérieur d'une boucle `[for]` peut créer plusieurs fichiers.

A7. Une query produit du texte ; un template calcule une valeur.

A8. Une modification faite à la main dans un fichier généré, hors zone protégée, est perdue à la régénération.

A9. Une ligne du template qui ne contient que `[/for]` produit une ligne vide dans le résultat.

A10. Dans `[file ('a.py', false, 'UTF-8')]`, la valeur `false` signifie « ne pas créer le fichier ».

Exercice 2 : Modèle ou méta-modèle ? [Simple]

Voici l'en-tête du module du tutoriel :

```
[comment encoding = UTF-8 /]
[module library2python('http://www.eclipse.org/emf/2002/Ecore')/]
[template public generateElement(root : EPackage)]
[comment @main/]
```

a) Que désigne l'URI `http://www.eclipse.org/emf/2002/Ecore` ? Pourquoi n'écrit-on pas l'URI du modèle `library` (`http://www.example.org/library`) ?

b) Pourquoi le paramètre `root` est-il de type `EPackage` ? Quelle est sa valeur quand on lance le générateur sur `library.ecore` ?

c) Compléter le tableau en indiquant, pour chaque élément, s'il appartient au **méta-modèle** (Ecore) ou au **modèle** (`library.ecore`) : `EClass`, `Book`, `eAttributes`, `title`, `EString`, `loan`, `containment`, `Member`.

d) `root.eClassifiers` contient-elle uniquement des classes ? Pourquoi le tutoriel écrit-il `->select(c | c.oclIsKindOf(EClass))` et le cours `->filter(EClass)` ?

Exercice 3 : Naviguer dans `library.ecore` [Simple]

On note `root` le paquetage, et `library`, `book`, `member`, `loan` les quatre `EClass`. Donner la **valeur** des expressions 1 à 8, puis écrire l'**expression** demandée aux lignes 9 à 13.

n°	Expression	Valeur / signification
1	`root.eClassifiers->filter(EClass)->size()`	
2	`root.eClassifiers->filter(EClass)->collect(c | c.name)`	
3	`book.eAttributes->size()`	
4	`book.eAttributes->first().name.toUpperFirst()`	
5	`member.eAttributes->select(a | a.lowerBound = 0)->collect(a | a.name)`	
6	`loan.eReferences->collect(r | r.eReferenceType.name)`	
7	`library.eReferences->forAll(r | r.containment and r.many)`	
8	`loan.eAttributes->exists(a | a.eAttributeType.name = 'EBoolean')`	
9		les attributs de type `EString` d'une classe `cls`
10		les classes qui ont au moins une référence
11		la classe nommée « Book »
12		les noms des classes référencées par Loan sans composition
13		le nom du getter Java d'un attribut `attr` (`getTitle`)

Question bonus. Que vaut `root.eClassifiers->filter(EClass)->collect(c | c.eAttributes->select(a | a.lowerBound = 1)->size())->sum()` ? Que calcule cette expression ?

Exercice 4 : Prédire le résultat d'un template [Simple]

On applique le module suivant à `library.ecore`.

```
[comment encoding = UTF-8 /]
[module resume('http://www.eclipse.org/emf/2002/Ecore')/]

[template public resume(root : EPackage)]
[comment @main/]
[file (root.name + '_resume.txt', false, 'UTF-8')]
Paquetage [root.name/] : [root.eClassifiers->filter(EClass)->size()/] classes
[for (cls : EClass | root.eClassifiers->filter(EClass))]
- [cls.name.toUpper()/] ([for (a : EAttribute | cls.eAttributes) separator(', ')][a.name/][if (a.
    lowerBound = 0)]?[/if][/for])
[for (r : EReference | cls.eReferences)]
   -> [r.name/] : [r.eReferenceType.name/][if (r.many)] ['[*]'/][/if][if (r.containment)] (
       composition)[/if]
```

```
[/for]
[/for]
[/file]
[/template]
```

a) Combien de fichiers sont produits ? Quel est leur nom ?

b) Écrire le contenu **exact** du fichier généré.

c) Pourquoi écrit-on `['[*]'/]` et non simplement `[*]` ?

d) Que deviendrait la ligne de Book si l'on supprimait `separator(', ')` ?

e) Que se passe-t-il si l'on oublie `[comment @main/]` ?

Exercice 5 : Compléter le générateur Python du tutoriel [Moyen]

On veut produire le fichier `library_model.py` : une classe Python par `EClass`, un constructeur dont les paramètres sont les attributs (valeur par défaut `None`), et une ligne d'initialisation par attribut et par référence. Compléter les trous ① à ⑥.

```
[comment encoding = UTF-8 /]
[module library2python('http://www.eclipse.org/emf/2002/Ecore')/]

[template public generateElement(root : EPackage)]
[comment @main/]
[file ('library_model.py', false, 'UTF-8')]
# -*- coding: utf-8 -*-
[for (cls : EClass | ① .........)]

class [② ....../]:
    def __init__(self[for (attr : EAttribute | ③ ......)], [attr.name/]=None[/for]):
[for (attr : EAttribute | cls.eAttributes)]
        self.[attr.name/] = ④ ......
[/for]
[for (ref : EReference | ⑤ ......)]
        self.[ref.name/] = ⑥ ......
[/for]
[/for]
[/file]
[/template]
```

a) Compléter les trous. Pour ⑥, on reprend d'abord la solution du tutoriel : une liste vide pour **toutes** les références.

b) Écrire le code produit pour la classe `Loan`. Quel défaut remarquez-vous par rapport au modèle ?

c) Écrire une query `initValue(ref : EReference) : String` qui renvoie `'[]'` si la référence est multiple et `'None'` sinon, et l'utiliser au trou ⑥.

d) Ajouter à chaque classe une méthode `__repr__` qui renvoie, par exemple pour Member : `Member(memberId='M1', name='Sara', email=None)`. (Indication : f-string Python `f"...{self.x!r}"`.)

Exercice 6 : Queries – la connaissance de la plateforme [Moyen]

Le tutoriel propose la query suivante :

```
[query public toSQLType(typeName : String) : String =
   if typeName = 'EString' then 'VARCHAR(100)'
   else if typeName = 'EInt' then 'INT'
   else if typeName = 'EBoolean' then 'BOOLEAN'
   else if typeName = 'EDate' then 'DATE'
   else 'VARCHAR(255)'
   endif endif endif endif
/]
```

a) Donner la valeur de `'EBoolean'.toSQLType()`, de `'EDouble'.toSQLType()` et de `attr.eAttributeType.name.toSQLType()` pour l'attribut `loanDate`.

b) Pourquoi y a-t-il quatre `endif` ? Pourquoi le dernier `else` est-il obligatoire ?

c) Ajouter le type `EDouble` → `DECIMAL(10,2)`.

d) Écrire sur le même modèle la query `toJavaType(typeName : String) : String` (EString → String, EInt → int, EBoolean → boolean, EDouble → double, EDate → Date, sinon Object).

e) Écrire la query `getterName(attr : EAttribute) : String` qui renvoie `isAvailable` pour un booléen et `getTitle` sinon.

f) Écrire la query `isPK(attr : EAttribute) : Boolean` : un attribut est **clé primaire** s'il s'appelle `id` ou si son nom se termine par `Id`. Quels sont les attributs clés de `library.ecore` ?

g) Pourquoi est-il préférable de ranger ces correspondances dans des queries plutôt que d'écrire des `[if]` directement dans le template ? Faire le lien avec le **PDM** de MDA.

Exercice 7 : Trouver et corriger les erreurs [Moyen]

Le module suivant contient **8 erreurs** (syntaxe ou logique). Les numéros de ligne sont donnés en commentaire. Pour chacune : donner la ligne, expliquer l'erreur et la corriger.

```
[comment encoding = UTF-8 /] 1
[module library2python('http://www.eclipse.org/emf/2002/Ecore')] 2
[template public generateElement(root : EPackage)] 3
[file ('library_model.py', 'UTF-8')] 4
[for (cls | root.eClassifiers->filter(EClass))] 5
class [cls.name]: 6
   def __init__(self[for (attr : EAttribute | cls.eAttributes)], [attr.name/]=None[/for]):
[for (attr : EAttribute | cls.eAttributes)] 8
       self.[attr.name/] = [attr.name/] 9
[/for] 10
[for (ref : EReference | cls.eAttributes)] 11
       self.[ref.name/] = [] 12
[/for] 13
[/file] 14
[/template] 15
```

Exercice 8 : Générer le script SQL [Moyen]

On veut produire `library.sql` avec les **règles** suivantes (version complète de l'exercice 1 du tutoriel) :

R1. une `EClass` → une table de même nom ;
R2. un `EAttribute` → une colonne, type obtenu par `toSQLType()` ; `NOT NULL` si `lowerBound > 0` ;
R3. la clé primaire est l'attribut `id` ou `...Id` (`isPK`) ; s'il n'y en a pas, on ajoute une colonne `id INT NOT NULL` en tête ;
R4. une `EReference` simple (`0..1`) et non contenante → une colonne `<nom>_id` du type de la clé de la table référencée, et une contrainte `FOREIGN KEY`.

On dispose des queries `toSQLType`, `isPK`, `hasPK(cls)`, `pkName(cls)` (nom de la clé, `'id'` par défaut) et `pkType(cls)` (type SQL de la clé, `'INT'` par défaut).

a) Écrire **à la main** l'instruction `CREATE TABLE` attendue pour `Member` puis pour `Loan`.

b) Compléter le corps de la boucle principale :

```
[for (cls : EClass | root.eClassifiers->filter(EClass))]

CREATE TABLE [cls.name/] (
[if (① ......)]
    id INT NOT NULL,
[/if]
[for (attr : EAttribute | cls.eAttributes)]
    [attr.name/] [② ....../][if (③ ......)] NOT NULL[/if],
[/for]
[for (ref : EReference | cls.eReferences->select(r | ④ ......))]
    [ref.name/]_id [⑤ ....../],
[/for]
    PRIMARY KEY ([cls.pkName()/])[for (ref : EReference | ...même collection...)],
    FOREIGN KEY ([ref.name/]_id) REFERENCES [⑥ ....../]([ref.eReferenceType.pkName()/])[/for]
);
[/for]
```

c) Expliquer pourquoi ce template ne produit **jamais** de virgule en trop avant la parenthèse fermante.

d) Pourquoi l'ordre des classes dans le modèle est-il important pour que le script s'exécute ?

e) On applique le même template à `school.ecore`, dans lequel la classe `Enrollment` a un attribut `grade : EDouble [0..1]` et deux références **obligatoires** (`1..1`) `student` (vers `Student`, clé `studentId : EString`) et `course` (vers `Course`, clé `courseId : EString`). Écrire la table produite. Proposer une modification pour que `student_id` et `course_id` soient `NOT NULL`.

f) **(Avancé)** Les compositions de Library ne sont pas traduites. Proposer une règle et la modifier dans le template pour ajouter, dans la table `Book`, une colonne `library_id` et sa clé étrangère vers `Library`. (Indication : `EReference.eOpposite` n'existe pas ici ; on cherche les références contenantes qui **pointent vers** `cls`.)

Exercice 9 : Générer les classes Java [Avancé]

On veut un fichier `library/<Classe>.java` par classe (exercice 2 du tutoriel), contenant : attributs privés, constructeur vide, constructeur complet, getters et setters, `toString()`. Le template principal est :

```
[template public generateElement(root : EPackage)]
[comment @main/]
[for (cls : EClass | root.eClassifiers->filter(EClass))]
[genClass(cls, root)/]
[/for]
[/template]
```

a) Où doit se trouver le bloc `[file]` pour obtenir un fichier par classe ? Écrire sa première ligne (le fichier doit être rangé dans un dossier portant le nom du paquetage).

b) Écrire le sous-template `genAccessors(attr : EAttribute)` qui produit le getter et le setter d'un attribut, en utilisant les queries `attrType(attr)` (type Java) et `getterName(attr)` de l'exercice 6. Donner le résultat pour `available`.

c) Écrire la ligne du template qui produit le **constructeur complet**, par exemple `public Member(String memberId, String name, String email) {`.

d) Écrire le texte Java produit pour la déclaration des **références** de `Library` et de `Loan`, sachant que le template contient :

```
[for (ref : EReference | cls.eReferences)]
    private [ref.refType()/] [ref.name/][if (ref.many)] = new ArrayList<>()[/if];
[/for]
```

avec `refType(ref) = if ref.many then 'List<' + ref.eReferenceType.name + '>' else ref.eReferenceType.name endif`.

e) Écrire la méthode `toString()` du template, de façon à produire pour Member :
`return "Member{" + "memberId=" + memberId + ", " + "name=" + name + ", " + "email=" + email + "}";`

f) Pourquoi est-il intéressant de découper le générateur en plusieurs templates (`generateElement`, `genClass`, `genAccessors`) ?

Exercice 10 : Documentation Markdown et zones protégées [Avancé]

Partie A — une nouvelle cible. On veut générer, à partir du **même** modèle, un fichier `library.md` :

```
# Modèle library (4 classes)

| Classe | Attributs | Références |
|---|---|---|
| Library | name | book -> Book, member -> Member, loan -> Loan |
| Book | id, title, isbn, author, available | |
| Member | memberId, name, email | |
| Loan | loanDate, returnDate | book -> Book, member -> Member |
```

a) Écrire le module complet `library2md.mtl`.

b) Faut-il modifier les générateurs Python, SQL et Java pour produire cette documentation ? Que montre cet exemple ?

Partie B – régénération. Le générateur Java produit, à la fin de chaque classe :

```
// Start of user code methodes Book
// Ajoutez ici votre code : il sera conservé àla prochaine génération
// End of user code
```

Un développeur ajoute la méthode `emprunter()` **entre** ces deux lignes, et corrige aussi « à la main » le getter `getTitle()` pour qu'il renvoie le titre en majuscules. On ajoute ensuite l'attribut `nbPages : EInt` à Book et on relance la génération.

c) Écrire les lignes du template qui produisent cette zone protégée.

d) Que contient `Book.java` après la régénération (nouvel attribut, `emprunter()`, `getTitle()`) ?

e) Quelle règle de travail faut-il donner à l'équipe ?

À retenir

- Un template Acceleo = le **texte final** dans lequel on a « percé des trous » `[expression/]`.
- Les types des variables (`EPackage`, `EClass`, `EAttribute`…) viennent du **méta-modèle** déclaré par l'URI du module ; le template marche pour **tout** modèle conforme.
- `[for]` répète, `[if]` choisit, `[file]` écrit, `[query]` calcule, `[protected]` protège.
- Méthode : écrire d'abord l'**exemple attendu** à la main, puis le transformer en template, petit pas par petit pas.